Class-70 04/08/2025

SPRING FRAMEWORK

* Struts → Spring Framework.
* This frameworks are used to do multiple works.
* 1. Spring core - basic setup of spring - module. (8) Context
  2. Spring JDBC - can do JDBC using spring.
  3. Spring ORM/ Spring Hibernate - can do ~~JDBC~~ Hibernate using Spring.
  4. Spring JPA - used to connect with database with ↓ advance features than Hibernate.
     (Java Persistance API)
  5. Spring MVC - Model View Controller (MVC)
     - Model ↓ POJO Classes
     - View ↓ HTML & JSP
     - Controller ↓ Servlet.
  6. Spring AOP (Aspect Oriented Programming)
  7. Spring Rest - helps to build API (Application Programming Interface)
     (SOAP)
  8. Spring Security - to handle authentication & authorisation. (Logging & signups).
  9. Spring AI

*

Spring Boot:

* It is also work with Spring.
* It is advanced version of Spring, much faster, less code & auto configuration (like Java to Java 8).
* It has integrated embadded server.
* It support spring Actuator - It supports some metrics.

* Caching:

Ex: client makes a call ⇆ (reached) server ⇄ Controller ⇄ DB (data)

(Caching) to
↳ store the data at some where. (temperory store).

* whenever data in DB updated, automatically caching also updated.

Class - 71

05/08/2025

* Design pattern followed by spring is Singleton.

* Singleton - Having single object for a class.

→ private constructor

**** Ex:

Project Name - Singleton

```
package com;
public class Employee {
    private static Employee emp;
    private Employee ( ) {

    }

    public static Employee getEmployee ( ) {
        return emp;
        if (emp == null) {
            emp = new Employee ( );
        }
        return emp;
    }
```

* Objects in spring is called Beans.

* In spring we never create a bean, due to, Spring handles beans.

* Spring handles beans using container (Store Beans)

→ Types - 2 - Containers

* 1. Bean Factory (depricated in latest versions) } Interfaces
  2. Application Context.

→ These interfaces hold containers.

* Implementation for that interfaces given in ClassPathXmlApplicationContext class. (xml file)

* For xml have a replacement with annotation in spring.

* AnnotationConfigApplicationContext - implementation given to interfaces when we use annotations.

```
public class Test {
    psvm {
        Employee employee = Employee.getEmployee();
        Employee employee2 = Employee.getEmployee();
        syso(employee == employee2); ↔ True.
    }
}
```

SPRING CORE / CONTEXT

* **IOC** - Inversion of Control (Reversing of Control)
  - → Inversing the control of beans.
  - → Spring works with IOC (i.e, spring handles / creates Beans).
  - →

* **Dependency Injection:**
  - → It is a heart of spring. It plays key role in it.

Ex: Employee → Has-A Relation → Address → Country → State → District → Mandal → Zipcode

⇒ Here employee class dependency on 6 other classes.

- → Null Pointer Exception.
- → To overcome this exception, have Depency Dependancy Injection. Spring handeles all dependencies & inject.
- → It is used to inject all dependencies in an object by Has-A relationship.

* Maven Project:
  - → Maven is a build tool that manages dependencies.

Ex: New → Mavenproject → Next → create a simple project → Next → GroupId (com) → ArtifactId (name) → Finish.

* pom.xml

<dependencies>

// to work with spring we need Spring Context.

Maven Repository → spring context.

<dependency>

<groupId> org.springframework </groupId>

<artifactId> spring-context </artifactId>

<version> 6.2.8 </version>

</dependency>

</dependencies>

src/main/java

Test class

```
package com;
public class Test {
    public static void main(String[] args) {
                                        import.
                                        (org.springframework.
                                            context)
        container                          xml
ApplicationContext ctx = new ClassPathXmlApplicationContext("beans.xml");
                                // Student st = new Student();
                                            ↑
// to get beans,
Student student = container.getBean("st", Student.class);
                                      ↓          ↓
                                    object      type.
                                    name
        output:
    syso(student); → [Hashcode]
        ↳ output: Student [id=1]

Student student2 = container.getBean("st", student.class);

                                    output:
                                    Student [id=1, name=Ravi]
                                    Student [id=2, name=FLM]
```

Student class

```
package com;

public class Student {
    private int studentId;
    private String name;
    public Student() {
                syso ("Student Object Created");
    }
        Getters, setters, toString.
    public student( int studentId, String name) {

    }
```

Output:
Student Object Created
~~com.Student@...~~
Student[id=1]

src/main/resources

New → xml file → Next → (beans.xml) → finish.

// to declare beans.

```
<?xml version = "1.0" encoding = "UTF-8" ?>
```

[Google ⇒ spring XSD]

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       :
       :

<bean class = "com.Student" > </bean>
              ↑ object name
<bean id="st" class = "com.Student" > </bean>
```

// to give studentId:

→ we can give with 1. property Tag (It calls setter to set value)
2. Constructor-Arg Tag (It calls constructor to set value).

```
<property name = "studentId" value = "1" > </property>
<property name = "name" value = "Ravi" > </property>
</beans>
```

(8)

```
<constructor-arg index="0" value="2" /></constructor-arg>
<constructor-arg value="FLM" index="1" /></constructor-arg>
</beans>
```

06/08/2025

class-72

* Spring work on ~~singletonstegi~~ design pattern. (funda)

Ex:

```
package com;
public class Employee{
    private int empId;
    private List<String> skills;
    private Map<Integer, String> hobbies;

    public Employee(){
    }
```

Constructor, Getters, setters, tostring.

beans.xml

```
<bean id="emp" class="com.Employee">
  <property name="empId" value="1"></property>
   <property name="skills">
        <list>
           <value>Java</value>
           <value>SQL</value>
            <value>JDBC</value>
            <value>Hibernate</value>
        </list>
    </property>
   <property name="hobbies">
      <map>
         <entry key="1" value="Cricket"></entry>
         <entry key="2" value="Badminton"></entry>
      </map>
    </property>
```

Test class:

```
package com;
public class Test {
        psvm {
  Application context container = new ClassPathXmlApplication
                                  Context("beans.xml");
Employee emp = container.getBean("emp", Employee.class);
        syso(emp);
syso(emp.getHobbies);
```

output:

Employee [empId=1, skills=[Java, SQL, JDBC, Hibernate]

{1=Cricket, 2=Badminton}.

Ex: Maven Project com.SpringCoreNoxml.

Class: src/main/java.

Dependency Injection: Here we used to refer dependencies manually.

Address Class:

```
package com;
public class Address {

    private int addrId;
    private string street;

    public Address() {
    }
    Constructor, Getter, setter, tostring.
```

Student class:

```
package com;

public class Student {

    private int stuId;
    private string name;
    private Address address;

    public student() {

    }
    Constructor, Getter, setter, to string.
```

beans.xml

// for address

```
<bean id="addressi" class="com.Address">
    <property name="addId", value="100"></property>
    <property name="street", value="Venkatala"></property>
</bean>

<bean id="st" class="com.student">
    <property name="stuId" value="1"></property>
    <property name="name" value="Ravi"></property>
    <property name="address" ref="addressi"></property>
```

ref → reference.

(~~Dynamic~~ Dependency Injection)

Test class:

```
package com;
public class Test {
        psvm {
ApplicationContext container = new ClassPathXmlApplicationContext("beans.xml");
Student student = containes.getBean("st", Student.class)
        syso(student);
        syso(student.getAddress());
```

Autowiring:

* It used to inject dependencies.
* It automatically inject dependencies.
* we can do it in 3 ways → 1. byType
  2. byName
  3. Constructor.

beans.xml

```
<bean id = "address1" class = "com.Address" >
  <property name = "addressId" value = "100" > </property >
  <property name = "street" value = "Venkatala > </property >
```

(search for address type)

byType:

```
<bean id = "st" class = "com.student" autowire = "byType" >
  <property name = "studId" value = "1" > </property >
  <property name = "studname" = streetname value = "Venkatala Ravi" > </property >
</bean >
```

byName:

```
<bean id = "st" class = "com.Student" > autowire = "byName" >
```

(It search for type name.)

```
  <property name = "studId" value = "100" > </property >
  <property name = "studname" value = "Ravi" > </property >
</bean.
```

Constructor type:

```
<bean id = "st" class = "com.Student" > autowire = "constructor"
```

(It search for construc-tor)

```
<property name = "studId" value = "1" > </property >
<property name = "name" value = "Ravi" > </property >
  <constructor-arg index=0 value = "1" > </constructor-arg >
  <constructor-arg index=1 value = "Ravi" > </constructor-arg >
</bean >
```

* byType (or) byName — when we use that, we have to use property tag. (<property></property>)

* If we want to use constructor type, we have to use constructor-arg tag. (<constructor-arg></constructor-arg>)

Ex: Maven Project SpringCoreNoxml.

Student class

```
package com;
@Component("students")
public class Student {
    @Value("${id}")
    private int stuId;
    @Value("${name}")
    private String name;
    @Autowired (field injection)
    private Address address;

    public ~~class~~ Student() {

    }
```

(@Autowired (Constructor injection)
Constructor, Getter, setter, tostring.

```
@Autowired (setter injection)
public void ~~setStudentId (int stuId)~~
        setAddr (Address addr) {
    this.addr = address
```

Address class:

```
package com;
@Component
public class Address {
    @Value("${id}")
    private int addrId;
    @Value("${street}")
    private String street;

    private Address() {

    }
```

constructor, Getter setter, tostring.

* @Autowired can be used on setter, field & constructor.
  → It by default uses byType autowiring.

beans.xml

<context:annotation-config></context:annotation-config> → (all annotations get enabled)

<bean id="st" class="com.Student" ~~autowire="byType"~~>

<property name="~~student~~Id" value="1"></property>

<property name="name" value="Gayatri"></property>

</bean>

<bean id="address" class="com.Address">

<property name="addrId" value="101"></property>

<property name="street" value="Gachibowli"></property>

</bean>

(2)

<bean id="st" class="com.student">

<property name="stuId" value="${id}"></property>

<property name="name" value="${name}"></property>

* <context:property-placeholder location="classpath:application.properties"/> (due to enable this we can use placeholders)

↳ placeholder

Test class

```
package com;

public class Test{

    psvm{

ApplicationContext container = new ClassPathXmlApplicationContext
                                ("beans.xml");

Student student = container.getBean("st", Student.class);

    syso(student);

    syso(student.getAddress);
```

ApplicationContext container = new AnnotationConfigApplicationContext (Config.class);

Src/main/resources

application.properties (properties file)

↳ It has all configuration details.
(like, url, password, username)

id = 1
name = "FLM"

// to create bean we have annotation,
1. autowire - @Autowired
2. bean - @Component (It alone will not create beans, it will create when it scans) - we have to enable in beans.xml.

beans.xml

<context:annotation-config></context:annotation-config>
<context:property-placeholder location="classpath:application.properties"/>

<context:component-scan base-package="com"></context:component-scan>

* placeholder - @Value.
(to read value from external file)
* beans.xml - @Configuration
* component scan = @ComponentScan
* @Qualifier - is used when has two beans, to give qualifying bean. It is used when what output
* @Primary - object we need.

// to remove beans.xml file.

// Configuration class.

```
package com;

@Configuration
@ComponentScan(basePackages = "com")
public class Config {
@PropertySource(name = "classpath:application.properties")

    public class Config {

    }
```

Package order example

→ com
→ com.model
→ com.controller
→ com.service
→ com.util
→ com.dao
→ com.dto

07/08/2025

Class-73 @Qualifier & @Primary

```
package com;

public interface Sim {

        void call();
```

```
package com;
@Component
public class Jio implements Sim {

    @Override
        public void call() {
            syso("Jio calling");
        }
```

```
package com;
@Component @Primary
public class Airtel implements Sim {

    @Override
        public void call() {
            syso("Airtel calling");
        }
```

```
package com;

public class Simtest {
        psvm {

Application-Context context = new AnnotationConfigApplication
                              Context(Config.class);
```

```
package com;
@Component
public class SimTest {

  @Autowired
//private Jio sim;
  @Qualifier("jio")
  private Sim sim;

  public void callSim() {
      sim
      jio.call();
  }
```

```
package com;
public class Test {

        psvm {

ApplicationContext container = new AnnotationConfigApplicationContext("
                                                    Config.class);

SimTest
simTest = container.getBean("simTest", SimTest.class);
    simTest.callSim();
  }

}
```

Bean Lifecycle.

1. Container is started
   ↓
2. Beans are initialised.
   ↓
3. Dependencies get injected.
   ↓
4. init (@PostConstruct) [method automatically calls even if we not call manually]
   ↓
   (Jakarta annotation dependency, should be write in pom.xml file.)
   → We can write only one @PostConstruct method, in one class.
   ↓
5. Normal Methods.
   ↓
6. Destroy (@PreDestroy)
   (registerShutDownHook() → deprecated.

Scopes of Beans ~~SPRING~~

→① All beans are in singleton mode (default)

→② Prototype - one class can have multiple objects (like Java)
- It creats new object (i.e, different object).

Ex:
```
package com;
@scope("prototype")
public class Users {

}
```

bean.xml (8)

```
<bean id="user" class="com.Users" scope="singleton">
<bean id="user" class="com.Users" scope="prototype">
```

~~package com;~~

~~public class Test {~~

③ request scope
④ session scope
⑤ application scope.

} applicable for web.

Class-74 SPRING JDBC 08/08/2025

* It creates connection on its own.
* we have to pass url and username & Password.
* It creates Statement
* we have to prepare Query & execute.
* It closes connection automatically.
* It provides JdbcTemplate - It bean (object) can call the things.
  @Bean (to create bean in Spring JDBC) - used with pre-defined classes.
  → we declare it in configuration class.
* To create JdbcTemplate, it needs DataSource.
* DataSource (Interface) will have url, username, password.
  → It has Has-A relationship - declared by using "ref" (Dependency Injection)
  → It provides DriverManagerDataSource (class) to implement.
* It handles checked exception & converts into unchecked exception (DataAccessException)

Ex: Maven Project - SpringJDBCXml

Dependencies

to search jdbc template.

beans.xml:

```
<bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
    <property id="" name="dataSource" ref="dataSource"></property>
</bean>
<bean id="dataSource" class="org.springframework.jdbc.core.DriverManagerDataSource">
    <property name="url" value="jdbc:mysql://localhost:3306/jfm-adv-java"></property>
```

ctrl+shift+t (80) ctrl+shift+r

```
<property name = "username" value = "root"></property>
<property name = "password" value = "Gayathrikausikumar@2000"></property>
</bean>
```

Class:

```
package com;

public class Test {

        psvm {

ApplicationContext context = new ClassPathXmlApplicationContext(
                                        "beans.xml");
JdbcTemplate
        template = context.getBean("jdbcTemplate", JdbcTemplate.class);

// Insert      template.update("Insert into employees values
                                        (?,?,?)",
                                    20, "abc@gmail.com", 10909);
```

(It accepts multiple parameters due to it has varargs parameter internally).

```
// update
        template.update("Update employees set email = ? where
                                        empId = ?", "bdc@gmail.com", 20);
// Delete
    template.update("Delete from employees where empId = ?", 20);
// select   (RowMapper - advanced version of resultSet)
                ↓
            (Interface) - FunctionalInterface.
Employee
    employee = template.queryForObject("select * from employees where
//anonymous class                 empId = 1", new RowMapper<Employee>() {
        @Override
        public Employee mapRow(ResultSet rs, int rowNum) {
        return new Employee(rs.getInt(1), rs.getString(2), rs.get
                                        Double(3));});
```

```
List<Employee> employee = template.queryForList("select * from employees", new
                RowMapper<Employee>() {
@Override
    public Employee mapRow(ResultSet rs, int rowNum) {
        return new Employee(rs.getInt(1), rs.getString(2), rs.getDouble(3));
    }
});          // to get all Employees details.
```

```
package com;
public class Employee {
    private int empId;
    private String email;
    private double salary;

    public Employee() {

    }
```

constructor, Getter, setter & tostring.

```
// with lambda
List<Employee> employee2 = template.query("select * from employees",
        (rs, num) -> new Employee(rs.getInt(1),
        rs.getString(2), rs.getDouble(3));

    Syso(employees2);
```

Ex: SpringJdbcNoXml – Maven Project.

→ Dependencies.

Employee class

Test class

```
package com;
public class Test {
        psvm {
ApplicationContext container = new AnnotationConfigApplicationContext("
                                                    Config.class");
JdbcTemplate template = container.getBean("template", "JdbcTemplate.class");
template.update("Insert into employees values(?,?,?), 2, "cdf@gmail.com",
                                                    25000);
```

.
.
.

```
package com;
@Configuration
  public class Config {
     @Bean("template")
     public JdbcTemplate getJdbcTemplate() {
       DataSource dataSource = getDataSource();
         return new JdbcTemplate(dataSource);

     }
     @Bean("dmds")
     public DataSource getDataSource() {
         return new DriversManagerDataSource("jdbc:mysql://
         localhost:3306/sjlm-adv-java", "root", Gayatriravikumar@2000");

     }

}
```